

Explicación del Algoritmo Módulo 5: Tendencia Acumulada Avanzada

1. Sección .data y .bss

Comenzamos definiendo en la sección .data todas las cadenas de texto estáticas necesarias para dar formato al archivo de salida. Posteriormente, declaramos el nombre del archivo resultante y reservamos espacio en la sección .bss para un *buffer* masivo donde ensamblamos todo el texto final antes de ejecutar la escritura en el archivo.

```
.data
// empezamos definiendo las cadenas estáticas para el archivo de salida
str_mod: .asciz "MODULE=ADVANCED_TREND\n"
str_tot: .asciz "TOTAL_VALUES="
str_inc: .asciz "INCREMENTS="
str_dec: .asciz "DECREMENTS="
str_mxu: .asciz "MAX_UP_STREAK="
str_mxd: .asciz "MAX_DOWN_STREAK="
str_acc: .asciz "ACCUM_DIFF="
str_tr: .asciz "TREND="
str_up: .asciz "UP\n"
str_dw: .asciz "DOWN\n"
str_st: .asciz "STABLE\n"
str_neg: .asciz "- "

// nombre del archivo de salida
file_out: .asciz "resultado_tendencia.txt"

.bss
// Buffer masivo para ensamblar el texto final antes de escribir al archivo
out_buf: .skip 2048
```

2. Etiqueta _start

Una vez declaradas nuestras variables y estructuras, procedemos a obtener el número de columna mediante los argumentos de ejecución (*command-line arguments*). Primero, verificamos si la cantidad de argumentos (argc) es menor a dos; de ser así, ejecutamos un salto condicional hacia la rutina de error. En caso contrario, accedemos a argv[1], que contiene el argumento de la columna en formato de texto (ASCII), y realizamos una conversión manual a entero (un algoritmo tipo *atoi/itoa*) para almacenar el valor numérico en el registro x11.

```

_start:
// =====
// obtenemos la columna via argumentos
// =====
ldr x0, [sp]          // Cargar cantidad de argumentos (argc)
cmp x0, #2
blt exit_err          // Si argc < 2, no se pasó columna, abortar

ldr x0, [sp, #16]      // Cargar puntero al string de argv[1]

// aqui nos preparamos para hacer un itoa manual del argumento de columna, ya que utils.s espera un número en x11
mov x11, #0            // x11 almacenará el número de columna parseado
mov x1, #10            // Multiplicador base 10
parse_col:
ldrb w2, [x0], #1      // Leer 1 byte de argv[1] y avanzar
cbz w2, col_parsed     // Si es nulo ('\0'), terminamos de leer
sub w2, w2, '0'        // Convertir ASCII a valor numérico
mul x11, x11, x1        // x11 = x11 * 10
add x11, x11, x2        // x11 = x11 + digito
b parse_col

```

3. col_parsed y Configuración del Stack

Si la columna se logra procesar con éxito, el flujo de ejecución llega a esta etiqueta.

Inmediatamente llamamos a la subrutina `utils_read_column_to_stack` de nuestra librería común `utils.s`. Esta función recibe el número de columna en `x11` y nos retorna dos direcciones de memoria fundamentales:

- **x0**: La dirección de inicio del *stack* (el tope superior).
- **x1**: La dirección final del *stack* original (el fondo).

A continuación, inicializamos en cero todos nuestros contadores (incrementos, decrementos y rachas). Luego, calculamos la posición inicial de nuestros datos restando 16 bytes a `x1` (el fondo del *stack*), y guardamos esta dirección en el registro `x4`. Dado que el *stack* en la arquitectura AArch64 debe estar alineado a 16 bytes, `x4` se convierte en un puntero a memoria que apunta exactamente al primer dato cronológico (`X_1`).

```

col_parsed:

// x11 ya contiene la columna. Llamamos a la rutina común.
bl utils_read_column_to_stack
// Retornos: x0 = Top del stack (X_30), x1 = Fondo del stack original

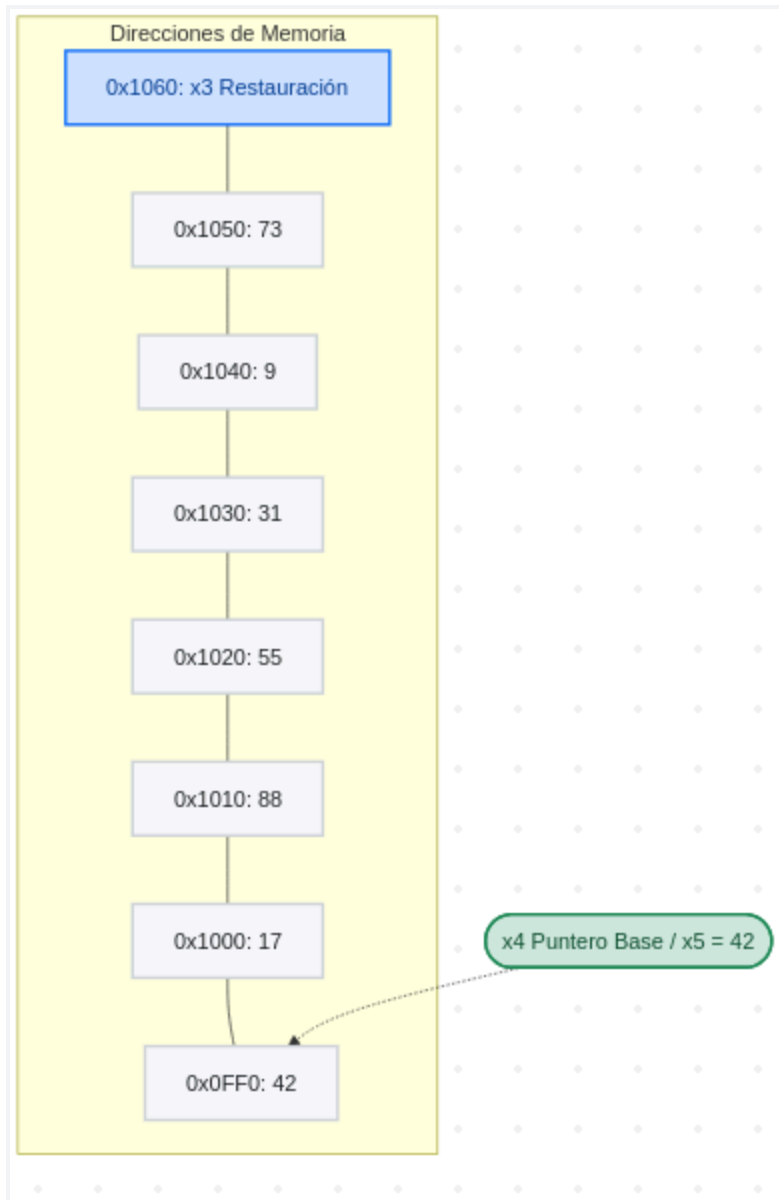
mov x19, #0           // Contador total de INCREMENTS
mov x20, #0           // Contador total de DECREMENTS
mov x21, #0           // Racha actual de crecimiento (Current Up)
mov x22, #0           // Racha MÁXIMA de crecimiento (Max Up Streak)
mov x23, #0           // Racha actual de decrecimiento (Current Down)
mov x24, #0           // Racha MÁXIMA de decrecimiento (Max Down Streak)

// El primer dato guardado está en el fondo del stack. Nos ubicamos ahí.
sub x4, x1, #16       // x_1

ldr x26, [x4]         // x26 = x_1
mov x25, x26          // x25 = x_1 (solo inicializando el "Anterior" para el primer ciclo)

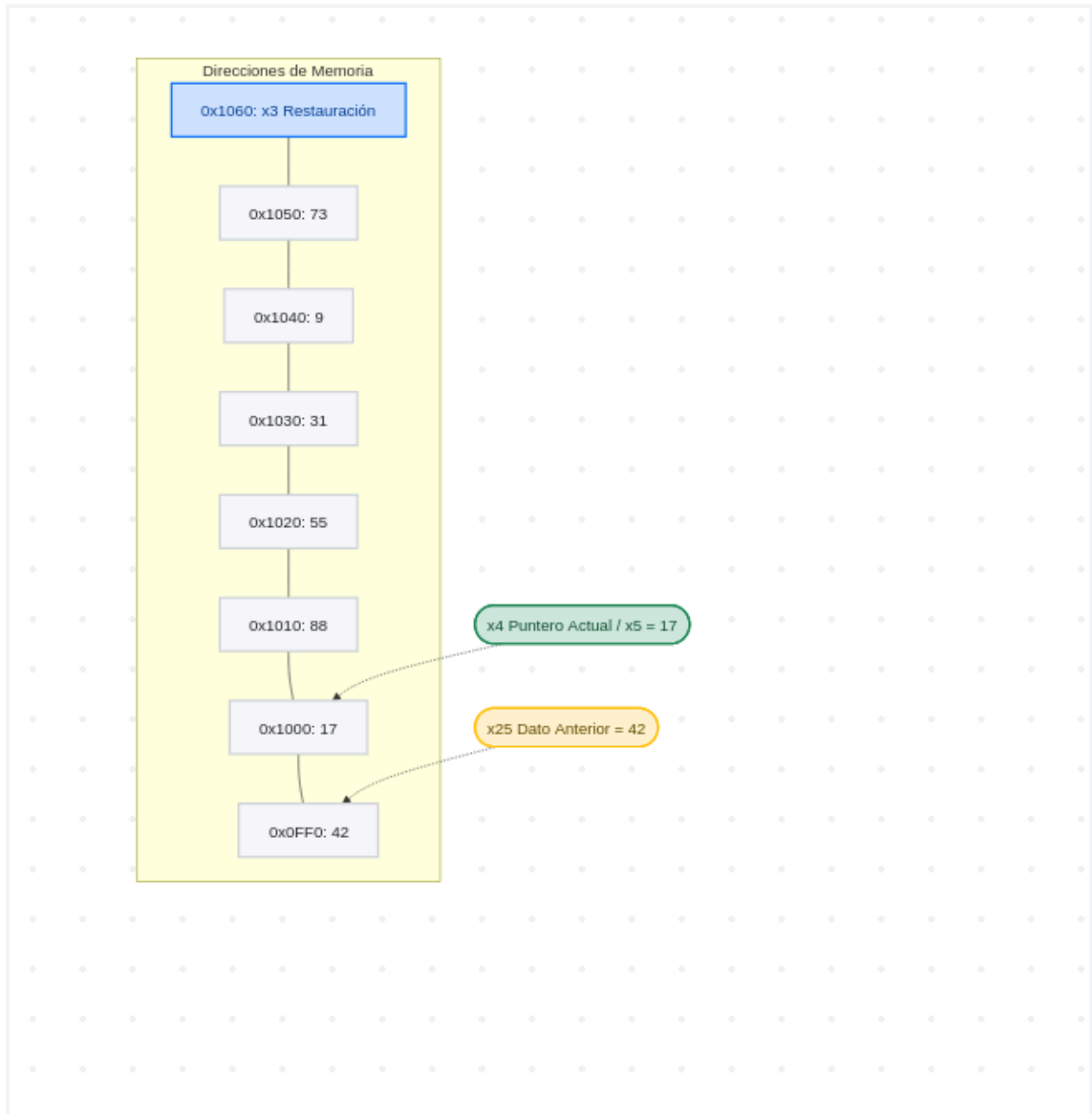
sub x4, x4, #16       // devolvemos el puntero a donde esta x_2

```



En este punto, procedemos a cargar en x26 el valor apuntado por x4 (en el ejemplo superior, el 42). Guardamos este valor inicial en un registro separado porque será indispensable al final para el cálculo matemático de la suma telescópica. Luego, copiamos este mismo valor a x25; esto nos sirve únicamente para establecer el estado del "Dato Anterior" antes de iniciar las comparaciones.

Finalmente, realizamos un paso crítico: restamos otros 16 bytes al puntero x4 para que ascienda a la siguiente dirección de memoria (0x1000 en el ejemplo). De esta forma, el puntero x4 se mueve al nuevo dato actual (17), mientras que x25 se queda resguardando el valor previo (42).



4. Ciclo Principal (loop_data)

```

loop_data:
    cmp x4, x0          // Comprobar si ya cruzamos el top del stack
    blt end_loop       // Si es menor, ya leímos los 30 datos

    ldr x5, [x4]        // Cargar Dato Actual (X_i)

    cmp x5, x25         // Comparar Dato Actual (X_i) con Dato Anterior (X_{i-1})
    bgt is_greater      // Si es mayor, saltar a lógica de incremento
    blt is_less         // Si es menor, saltar a lógica de decremento

```

```

is_equal:
    // Si son iguales, se rompen ambas rachas actuales
    mov x21, #0
    mov x23, #0
    b next_iter

```

```

is_greater:
    add x19, x19, #1    // INCREMENTS++
    add x21, x21, #1    // Racha actual de subida++
    mov x23, #0         // Romper racha de bajada

    cmp x21, x22        // ¿Racha actual supera a la racha máxima registrada?
    ble next_iter
    mov x22, x21        // MAX_UP_STREAK = racha actual
    b next_iter

```

```

is_less:
    add x20, x20, #1    // DECREMENTS++
    add x23, x23, #1    // Racha actual de bajada++
    mov x21, #0         // Romper racha de subida

    cmp x23, x24        // ¿Racha actual supera a la racha máxima registrada?
    ble next_iter
    mov x24, x23        // MAX_DOWN_STREAK = racha actual

```

A partir de aquí entramos al ciclo iterativo principal. Lo primero que ejecutamos es una comprobación de límites: comparamos si x4 (nuestro puntero actual) es menor a x0 (el tope del *stack*). Si el puntero ha cruzado este límite, significa que ya procesamos los 30 datos correspondientes y ejecutamos un salto incondicional hacia la etiqueta `end_loop`. Si aún quedan datos por leer en la memoria, cargamos en el registro x5 el valor referenciado por el puntero x4 (nuestro VALOR ACTUAL). Posteriormente, comparamos el valor en x5 (actual) con el registro x25 (anterior) para determinar la naturaleza de la tendencia.

Condición de Bifurcación	Lógica Ejecutada
is_equal (Son Iguales)	Se interrumpen ambas rachas. Reiniciamos los registros contadores de las rachas actuales a 0, y saltamos hacia <code>next_iter</code> para actualizar los punteros y retornar al inicio del

Condición de Bifurcación	Lógica Ejecutada
	ciclo.
is_greater (Es Mayor)	Incrementamos el contador general de subidas y la racha actual de crecimiento. Ponemos a cero la racha actual de bajada. Luego, evaluamos si esta racha actual supera el récord máximo almacenado en nuestro registro de máximos; de ser así, lo sobrescribimos y registramos un nuevo máximo histórico.
is_less (Es Menor)	Aplicamos exactamente la misma lógica operacional que en is_greater, pero incrementando los contadores de decremento y validando contra el registro encargado de mantener el récord máximo de rachas de bajada.

La etiqueta `next_iter` se encarga de reconfigurar los parámetros para la iteración entrante: actualizamos nuestro "valor anterior" (x25) asignándole el "valor actual" (x5) y volvemos a restar 16 bytes al puntero x4 para avanzar secuencialmente al siguiente dato en la pila. Finalmente, retornamos al `loop_data`.

5. Suma Telescópica y Construcción de la Salida (`end_loop`)

```
end_loop:
    // Suma Telescópica ->  $\Sigma(X_i - X_{i-1}) = X_{30} - X_1$ 
    sub x27, x25, x26    // x27 = ACCUM_DIFF
```

Al concluir la iteración por todos los datos de la columna, aplicamos un artificio matemático utilizando una suma telescópica para calcular la diferencia acumulada (**ACCUM_DIFF**) de manera altamente eficiente. En lugar de procesar la sumatoria componente por componente, simplemente restamos el último dato procesado (alojado en x25 tras el fin del ciclo) menos el primer dato de la serie (el que habíamos respaldado estratégicamente en x26 al inicio de la rutina).

A continuación, procedemos a ensamblar nuestro archivo de texto concatenando los resultados numéricos y cadenas en nuestro *buffer* masivo, apoyándonos en las subrutinas locales

append_string y append_number. Para las lógicas con signo, evaluamos **ACCUM_DIFF** dinámicamente; si el resultado es negativo, anexamos explícitamente el carácter "-" al *buffer* e invertimos el signo del número (calculamos su valor absoluto) para que append_number realice el *itoa* sin interrupciones.

6. Escritura en el txt y Finalización (write_file)

```
write_file:
    // Calcular longitud exacta del buffer dinámico creado
    ldr x0, =out_buf
    mov x1, #0
len_loop:
    ldrb w2, [x0, x1]      // Leer byte
    cbz w2, exec_write     // Si es nulo, terminamos de contar
    add x1, x1, #1        // Longitud++
    b len_loop

exec_write:
    // x0 ya tiene la direccion del buffer de texto
    // x1 ya tiene la longitud calculada
    ldr x2, =file_out      // x2 recibe el nombre del archivo final
    bl utils_write_result  // Llamada al syscall 64 de utilidades

    // =====
    // salida limpia
    // =====
    mov sp, x3             // Restaurar el stack a como estaba antes de leer CSV

exit_err:
    mov x0, #0
    mov x8, #93            // Syscall: exit
    svc #0
```

Para la fase de escritura final, establecemos el registro x0 apuntando directamente a nuestro *buffer* de salida ya construido. Empleamos un bucle pequeño para recorrer el *buffer* iterativamente byte a byte hasta localizar el carácter nulo de final de cadena (`\0`), llevando el conteo en x1 para calcular la longitud exacta de los datos.

Una vez validada la longitud, bifurcamos a `exec_write` donde delegamos la tarea a la utilidad de escritura del archivo común `utils.s`. Colocamos el nombre de salida en x2, la dirección del *buffer* en x0 y su longitud en x1. Como paso final e imperativo, restauramos el estado del *stack* original recuperando el registro base de restauración (x3) antes de ejecutar la llamada al sistema operativa (*syscall* de salida) y así terminar el proceso limpiamente.

7. Subrutinas Locales


```

append_string:
    ldr x1, =out_buf
find_end:
    ldrb w2, [x1]           // Leer byte del destino
    cbz w2, copy_str        // Si encontramos el '\0', empezamos a copiar acá
    add x1, x1, #1
    b find_end
copy_str:
    ldrb w2, [x0], #1        // Leer byte del origen
    strb w2, [x1], #1        // Escribir byte en destino
    cbnz w2, copy_str        // Repetir hasta que el origen mande su '\0'
    ret

```

- **append_string:** Cargamos el *buffer* de destino en x1 y lo recorremos secuencialmente hasta detectar su propio fin de cadena (\0). Una vez ubicado el último byte válido, comenzamos un proceso de copiado iterativo desde el *buffer* de origen (x0) hacia el destino, byte a byte, que se interrumpe solo cuando el origen mismo entrega un byte nulo.

```

append_number:
    stp x29, x30, [sp, #-16]! // Guardar link register
    ldr x1, =write_buffer      // Buffer temporal para itoa
    bl utils_itoa              // utils_itoa genera "numero\n\0"

    ldr x0, =write_buffer      // Tomar el resultado de itoa
    bl append_string           // Enviar al concatenador general

    ldp x29, x30, [sp], #16    // Restaurar link register
    ret

```

- **append_number:** Comienza con un prólogo convencional para funciones anidadas, resguardando el registro de enlace (Link Register - x30) y el *frame pointer* (x29) en el *stack*. Se sirve de la subrutina *itoa* de nuestra librería para hacer la conversión numérica a representación ASCII sobre un *buffer* temporal. Luego, simplemente enviamos el puntero de ese resultado temporal a nuestra función `append_string` para añadirlo al bloque principal, finalizando el proceso con el respectivo epílogo de restauración.

EVIDENCIA DE DEPURACIÓN VÍA GDB

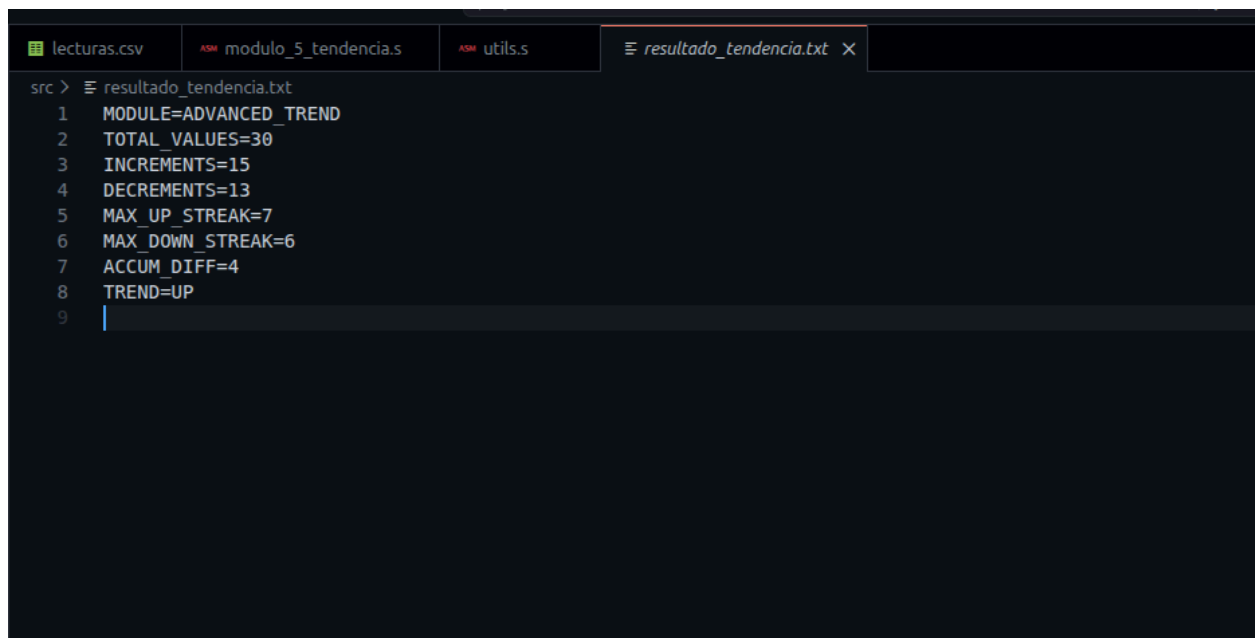
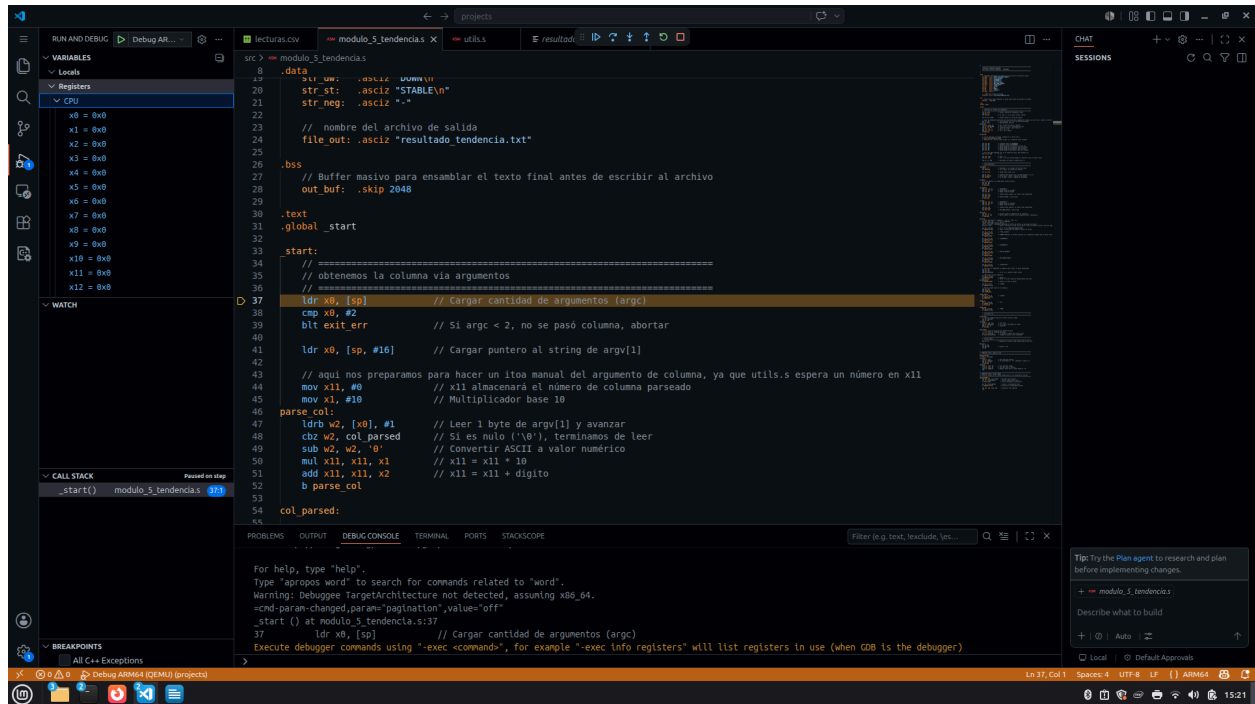


DIAGRAMA DE FLUJO

